

SMART NURSERY GUARDIAN

An AI-Assisted Embedded Monitoring System for Infant Care

Project: Final Project Report

Team: MindCloud — Team 10

Institution: Alexandria University, Faculty of Engineering

Department: Mechatronics / Electromechanical Engineering

Date: September 2026

Version: 1.0 — Release

1

PROJECT OVERVIEW

Smart Nursery Guardian addresses one of parenthood's most exhausting challenges: not knowing why an infant is crying. At 3 AM, with no contextual clues, distinguishing a hungry cry from a tired cry or a discomfort cry is nearly impossible. The system solves this by listening to the cry, classifying its cause using machine learning, and responding automatically — rocking the crib, adjusting lighting and cooling, and alerting caregivers to safety hazards. It operates like an attentive caregiver that never sleeps and never misses a signal.

The system combines machine learning running on a laptop with a real-time embedded microcontroller to monitor the nursery environment and react intelligently. Sensors track temperature, humidity, motion, light levels, and gas concentration. Actuators respond with crib rocking, cooling, lighting, and alerts — all driven by the classification result.

Why This Project Matters

- Addresses sleep loss from misinterpreting infant cries — turning guesswork into data-backed guidance
- Realistic technical scope: signal processing, machine learning, real-time firmware, GUI design, and IoT
- Demonstrates the standard edge-device + smart-device IoT architecture used professionally

System Components

Component	Location	Responsibility
Python App (Tkinter)	Laptop	Mic input, noise filter, cry detection, ML classification, GUI, Telegram alerts
ESP32 Firmware	Microcontroller	PIR, light, temperature, gas sensors; servo, buzzer, LED, fan control
Telegram Bot	Cloud (free)	Gas alert notifications to caregiver phone from any location

Split architecture rationale: AI and audio processing require Python's libraries and compute power. Real-time sensor and motor control require the MCU's low-level timing. Audio never crosses serial — giving zero-latency cry detection. Other sensors relay through the MCU over UART.

2 HARDWARE & PCB DESIGN

The hardware platform centers on the ESP 32 Developer Board. The PCB uses STM32F401CEU6 (UFQFPN-48, QFN 7x7mm, 0.5mm pitch) and a 4-layer JLCPCB-ready board transitioning the breadboard prototype into a manufacturable, assembly-ready product with full supplier information on every component.

STM32F401CEU6 — Pin Assignment

Pin	Signal	Type	Connected To
PA0	DHT11 Data	GPIO 1-Wire	DHT11 + 4.7k pull-up to 3.3V
PA1	LDR ADC	ADC1_IN1	LDR voltage divider (10k)
PA2	Gas Sensor	ADC1_IN2	MQ-x AOUT via 10k/20k divider
PA3	PIR Signal	GPIO Input	HC-SR501 digital output
PA4-PA7	RGB LED	GPIO Output	R/G/B channels via 330 ohm resistors
PA6	Servo PWM	TIM3_CH1	3-pin servo header (+5V)
PA9 / PA10	UART TX/RX	USART1	Laptop serial link (J7 header)
PA13 / PA14	SWD Debug	SWD	5-pin debug header (J2)
PB0	Fan PWM	TIM3_CH3	L293D ENABLE1 pin (fan speed)
PB1	Room LED	GPIO Output	330 ohm to 2-pin header J_LED
PB10	Buzzer	GPIO Output	3.3V active buzzer (direct drive)
PB12 / PB13	Motor Direction	GPIO Output	L293D IN1 / IN2

PCB Layer Stackup (4-Layer, 1.6mm, JLCPCB)

Layer	Name	Type	Net Assignment
L1 (Top)	Top Signal	Signal	Component placement + signal routing
L2	GND Plane	Plane	Solid GND pour — no signal routing here
L3	Power Plane	Plane	+3V3 and +5V copper pours (split plane)
L4 (Bottom)	Bot Signal	Signal	Secondary signal routing

JLCPCB Design Rules Applied

Rule	Value	Rule	Value
Min trace width	0.2mm	Min via hole	0.3mm
Min clearance	0.2mm	Via outer diameter	0.6mm

USB diff. width	0.2mm	USB diff. gap	0.2mm
Board edge clearance	0.3mm	Solder mask expansion	0.051mm
Copper weight (outer)	1 oz	Copper weight (inner)	0.5 oz

3 MACHINE LEARNING PIPELINE

The ML subsystem runs entirely in Python on the laptop and classifies each detected cry as Hungry, Tired, or Discomfort. The trained model is loaded once at startup and classifies every new cry in real time without retraining. Audio is read directly from the laptop microphone and never crosses the serial link, ensuring zero-latency cry detection.

3.1 — Audio Processing Chain

Stage	Tool / Method	Purpose
1. Capture	PyAudio (rolling stream)	Continuous microphone input in chunks
2. Noise Reduction	noisereduce library	Spectral gating to remove hum, fans, background noise
3. Voice Activity Detection	webrtcvad library	Confirm a real cry is present — avoid false triggers
4. Feature Extraction	librosa	Extract numerical features from confirmed cry segment
5. Classification	scikit-learn	Random Forest predicts Hungry / Tired / Discomfort
6. Response	Tkinter GUI + PySerial	Display result, trigger hardware actions over UART

3.2 — Feature Extraction (Current)

Feature	Description	Output Dim.
MFCC (13 coefficients)	Tone color and spectral shape — mimics human auditory perception	26 (mean+std)
Fundamental Frequency (F0)	Base pitch — infant cries range 400-600 Hz typically	2 (mean+std)
RMS Energy	Average loudness and intensity of the cry	1
Zero-Crossing Rate	Harshness and noisiness of the signal	2 (mean+std)
Duration	Total length of the confirmed cry segment in seconds	1

Start with MFCC alone — it performs close to best-published results by itself. Add other features and measure if accuracy actually improves. Do not add complexity without proof of benefit from the confusion matrix.

4 THE CLASS IMBALANCE PROBLEM

The Donate-A-Cry corpus (457 recordings) has a severe class imbalance that reflects reality — hungry cries genuinely occur more often than tired or discomfort cries. However, training a classifier directly on this raw distribution produces a model that is statistically impressive but practically useless: it learns to predict Hungry for everything and still achieves 89% accuracy.

Dataset Distribution

Class	Recordings	Percent	Problem
Hungry	382	84%	Model defaults to this — inflates overall accuracy
Discomfort	27	6%	Severely under-represented in training
Tired	24	5%	Nearly invisible to the classifier

Current Model Results

The model achieves 89.66% overall accuracy — but this number is entirely misleading:

Class	Precision	Recall	F1-Score	Support	Assessment
Hungry	0.90	1.00	0.94	77	Over-predicted
Discomfort	1.00	0.20	0.33	5	Severely under-detected
Tired	0.00	0.00	0.00	5	NOT DETECTED AT ALL
Accuracy			0.90	87	
Macro Average	0.63	0.40	0.43	87	Critical failure point
Weighted Average	0.85	0.90	0.86	87	

Why 89.66% accuracy is misleading: a trivial model that always predicts "Hungry" scores 88.5%. The macro-average F1 of 0.43 is the honest metric — it weights all classes equally. Tired cries are never detected (F1 = 0.00) and Discomfort is barely caught (F1 = 0.33). Overall accuracy is not valid here.

Data Augmentation — Applied Solution

Augmentation creates synthetic variants of minority-class recordings using acoustically valid transformations, applied to training data only. The test set is never augmented to preserve evaluation integrity.

Technique	Parameters	Why It Helps
Pitch Shift	+2 semitones maximum	Prevents memorizing exact frequency signatures — pitch-invariant patterns
Time Stretch	0.9x to 1.1x speed (pitch preserved)	Teaches cry-length invariance across different playback speeds

Noise Mixing	Light background noise overlay	Improves robustness to room acoustics and microphone variation
--------------	--------------------------------	--

Split 80/20 before any augmentation begins. Augment only training data. Always evaluate using a per-class confusion matrix — not overall accuracy.

The core limitation of the current pipeline is that it discards all temporal structure. Summarizing an entire cry clip into a single mean-and-std vector loses how the cry evolves over time — yet that temporal evolution is precisely what distinguishes cry types. A tired cry starts strong and fades; a hungry cry escalates. None of this is captured in global statistics.

5.1 — 40ms Frame Extraction

Instead of one feature vector per clip, one MFCC frame is extracted every 40ms with 50% overlap. A 3-second cry produces approximately 75 frames — a 2D array of (time_steps x features) that preserves the temporal evolution of the cry. This is the fundamental change that unlocks the deeper architecture.

5.2 — CNN — Spatial Feature Extractor

1D convolutions across the time dimension detect local spectral micro-patterns: sharp onset, sustained vowel, rapid decay. These short-term acoustic fingerprints are richer and more discriminative than global MFCC statistics. The CNN processes each 40ms frame and outputs a learned feature representation for every time step in the sequence.

5.3 — LSTM — Temporal Context Modeling

The CNN-processed sequence feeds an LSTM which maintains a hidden state across all time steps, learning long-range dependencies — how the cry evolves over seconds. A tired cry that starts strong and fades produces a fundamentally different LSTM hidden state than a hungry cry that builds in intensity, even when individual frames look similar. This is the critical capability the current pipeline entirely lacks.

5.4 — Attention — Diagnostic Frame Selection

Not all 40ms frames are equally informative. The Attention layer assigns a learned weight to each time step — frames containing class-discriminative patterns receive high weight; noise frames receive near-zero weight. The final classification uses a weighted sum of LSTM outputs, making the model robust to cry length variation and to the positional differences in diagnostic moments between minority-class samples.

Architecture Comparison

Factor	Current (Random Forest)	Proposed (CNN + LSTM + Attention)
Input representation	Single vector per clip	Sequence of 75+ 40ms frames
Temporal information	None — discarded entirely	Fully preserved via LSTM hidden state
Feature learning	Manual (MFCC statistics)	Learned automatically by CNN layers
Frame importance	All frames equal weight	Attention assigns weight by relevance
Augmentation benefit	Limited (global stats shift)	High — diverse frame-level sequences
Expected macro F1	0.30 - 0.43 (current result)	0.75+ (published literature target)
Training requirement	Seconds on CPU	Minutes on GPU or Google Colab free tier

Laptop Main Loop (Python)

Step	Action	Detail
1	Capture	Continuous microphone audio in rolling chunks via PyAudio
2	Filter	Spectral noise reduction using noisereduce library
3	VAD	webrtevad confirms real cry is present — prevents false triggers
4	Signal MCU	Send CRY_DETECTED over UART — servo starts immediately
5	Classify	Extract features and run through trained ML model
6	Respond	Hungry: play video. Tired: sound buzzer. Discomfort: show message.
7	Clear	Send CRY_ENDED when cry signal drops — stop servo

Serial Message Protocol (UART 9600 baud)

Direction	Message	Trigger	Action Taken
MCU -> Laptop	BABY_AWAKE	4+ PIR events in 8 seconds	Update GUI baby-awake state
MCU -> Laptop	GAS_ALERT	Gas sensor threshold exceeded	Show red full-screen alert + send Telegram
MCU -> Laptop	TEMP:<val>	Periodic every 5 seconds	Update temperature display in GUI
Laptop -> MCU	CRY_DETECTED	VAD confirms cry present	Start servo + enable LED cry-state logic
Laptop -> MCU	CRY_ENDED	Cry signal drops	Stop servo + clear LED cry-state
Laptop -> MCU	BUZZER_ON	Tired cry classified	Sound 3.3V active buzzer immediately

Temperature Control Response Table

Temperature Range	RGB LED Color	Fan Speed	PWM Duty Cycle
Below 25 deg C	Blue	OFF	0%
25 deg C to 30 deg C	Green	Half Speed	50%
Above 30 deg C	Red	Full Speed	100%

Fan and RGB LED are fully local decisions (temperature only — no laptop input needed). The room LED is the exception: it runs on the MCU but depends on cry-detected state from the laptop over UART.

7

SUMMARY & PROJECT STATUS

Smart Nursery Guardian successfully integrates machine learning, embedded systems, IoT communication, and real-time firmware into a cohesive infant monitoring platform. The 4-layer STM32F401CEU6 PCB is complete and DRC-validated. The Python pipeline achieves real-time cry classification and environmental monitoring with immediate hardware response across all sensor and actuator channels.

The primary open challenge is the macro-average F1 score of 0.30 to 0.43, caused entirely by class imbalance in the Donate-A-Cry corpus. The current Random Forest classifier fails to detect Tired cries (F1 = 0.00) and barely detects Discomfort (F1 = 0.33) despite 89.66% overall accuracy. The solution is fully defined: CNN + LSTM + Attention operating on 40ms frames, combined with continued data augmentation on minority classes.

Deliverables Status

Deliverable	Status	Notes
STM32 Schematic (6 sections)	Complete	DRC clean, all nets labeled and cross-referenced
4-Layer PCB Layout	Complete	JLCPCB rules verified, DRC passed
Sensor and Actuator Firmware (HAL)	Complete	Breadboard prototype assembled and tested
Python ML Pipeline — RF Baseline	Complete	Deployed and classifying in real time
Data Augmentation	Complete	Pitch shift, time stretch, noise mixing applied
Telegram Gas Alert Simulation	Complete	BluePill simulation running in Proteus
CNN + LSTM + Attention Architecture	Proposed	Architecture fully defined, pending implementation

With the proposed CNN + LSTM + Attention upgrade and continued minority-class augmentation, the macro-average F1 score is expected to exceed 0.75 — transforming the system from a statistically accurate but class-biased tool into a genuinely reliable infant cry classifier that correctly detects Tired and Discomfort cries under real-world data imbalance.